

1) Difference between let, var, & const.

var

"var" is a traditional way of making a variable, it can declare, initialize, redeclare, reassign in a functional scope.

let

"let" is introduced in ES6, that we can declare, initialize but can't redeclare & reassign and it is a block scope.

const

"const" also introduced in ES6, that can only initialize the value cannot declare, redeclare, and reassign.

2) What's Hoisting:

Hoisting is a Javascript behaviour of moving the variable & function declaration to the top of the scope during the memory creation phase before the execution.

```
console.log(a);
```

```
var a = 10;
```

output is : undefined.

it works for var as "undefined", but const & let it works as ReferenceError because of Temporal Death Zone (TDZ).

3) TDZ.

TDZ is a Temporal Death Zone. is time between the variable declaration, initialization & cannot access let, const before the initialization the output is ReferenceError: cannot access "VariableName".

```
console.log(name)
let name = "8-urlc";
```

4) What is Execution Context.

Execution context is an environment of where the Javascript code runs. There are two phases:

Memory creation phase,

Execution phase.

Memory creation phase & Execution phase.

The memory creation phase is that allocate the memory space to variable and function. var is initialized with undefined, let & const are placed TDZ and function declaration are stored completely.

In execution phase, the run line by line and update the value allocate memory.

Three types of Execution Context.

Global Execution context

Function execution context

Eval execution context.

5) Scope:

Scope determines where the variable access in a program. Four types

* Global scoping → the variable can access in outside of the function.

eg:

```
let Name = "Guru";  
const username = () => {  
  console.log("Name", Name);  
}  
username();
```

* Function scoping → the variable can access inside the function.

eg:

```
function greet = () => {  
  let message = "Hello";  
  console.log(message)  
}
```

~~function greet()~~;
console.log(message) → cannot access.

* Block scoping → the variable can access in {} inside the block. (let & const).

```
if (age) {  
  let age = 20;  
  console.log(age);  
}
```

console.log(age) → cannot access.

b) lexical scope: chain scope.

lexical scope means a function can access the variable from its own function or its other all outer function.

chain scope → when JS looking for variable it search from current scope, parent scope, Grandparent scope & Global scope.

eg:

```
let company = "Objectways";  
function outside() {  
  let team = "Sourcing";  
  function inner() {  
    let project = "Videosourcing";  
    console.log(project);  
    console.log(team);  
    console.log(company);  
  }  
  inner();  
}
```

Videosourcing
Sourcing
objectways.

7) what is closure, real world example.

closure function is created when the inner function remember, and continuous to access the outer function variable even the outer function finish the execution.

```
const counting = () => {  
  let count = 0;  
  return Counter() {  
    count++;  
    console.log("counter:", count);  
  }  
  return counter;  
}
```

```
const result = counting()  
result();
```

8) closure preserve variable

Normally, when the function finish it executes it delete the memory. but in closure function the inner function doesn't delete the memory, still remains the variable value of memory.

9) call back function :

call back function is passing the argument to the another function.

```
eg:- const message = (cb) => {  
  console.log("Hello")  
  cb();  
}
```



```
const goodMessage = () => {
  console.log("I'm happy")
}
```

message(goodMessage);

Why we use callback function.

callback function used because of some operations we take time to complete, JS doesn't wait for task complete. so callback is used to control the task timing, like APIs calls, database operations, Event handling, Timers & Array methods.

b) callback injection:

callback injection means passing different callback function into same function and allowed to decide what operation to execute.

```
const processOrder = (callback) => {
  console.log("order Received");
  console.log("processing order");
  callback();
}
```

```
const mobileOrder = () => {
  console.log("mobile is ordered")
}
```

processOrder(mobileOrder)


```
const ApplePhone = () => {  
  console.log("ApplePhone ordered")  
}
```

```
processOrder(ApplePhone);
```

1) Different between callback & closure.

Callback:

A function passed as an argument to another function.
used to execute code later.

Focuses on when a function is executed.

Closure:

A function that remembers variable from its outer scope.

Scope:

used to preserve data

Focuses on what variable a function can access.

2) Different between Rest Parameter & Spread Operator.

Rest Parameter:

Collects multiple values into a single array.

used in function parameters

Receives values

eg. ~~const sum~~ ^{const} ~~sum~~ = (...numbers) => {
 console.log(numbers);
}

```
sum(10, 20, 30, 40);
```


Spread operator

Expands an array or object into individual values using in function calls, array, and object.
unpack values.

eg: `const numbers = [10, 20, 30];`
`console.log(...numbers);`

13. difference between return & console.log().

The return statement is used to send a value back from a function to place where the function was called. The returned value can be stored in variable, used in calculation and passed to another function.

The console.log() is used only to display information in the browser's console. for debugging or testing purpose. It doesn't send any value back to function.

14) Truthy & falsy.

A truthy value is any value that evaluates to true, and a falsy value is any value that evaluates to false. most values in javascript are truthy including non-empty string, non-zero numbers, arrays, objects & function.

falsy value

false, 0, -0, "", "", NaN, null, undefined, NaN

15) Shallow copy & Deep copy.

A shallow copy creates a new object but shares references to nested objects with the original object.

A deep copy creates a completely independent copy of the object, including all nested objects and array. Therefore's changes in deep copy don't affect the original, but shallow copy to nested data can affect both the original & copy.

16) Differences between map(), filter(), reduce()

map()

map() method is used to transform every element of an array and returns a new array of the same length.

filter()

filter() method is used to select the element which satisfy the condition & returns a new array containing those element.

reduce()

reduce() is used to combine all elements of an array into single value, such as product, sum or objects.

17) forEach() and map()

forEach() method executes a function for each element in the array and doesn't return a new array.

map() method is used to transform every element of an array & return new array of the same length.

18) find() & filter()

The find() returns only first matching element, but filter() returns all matching elements in a new array.

19) for...of, for...in & forEach.

for...of loop is used to iterate over the values of iterable objects such as arrays and strings.

for...in loop is used to iterate over the keys (properties) of an object.

forEach() method is an array method that executes a callback function in each element in array.

20) Object Destructuring, Enhance Object Literals.

They are modern Javascript features that make code shorter, cleaner and easier to read. They are heavily used in React, because we work extensively with props, state & objects.

Object Destructuring: that allows to extract properties from an object and store them in variables.

Enhance object literals: Shortway to create object when variable names and property names are the same.